

4.13 - Zero trust, service mesh e secrets

Nunca confie, sempre verifique

Zero trust: a rede não é confiável

Identidade vira o novo perímetro

- **Nunca confie, sempre verifique cada acesso**
 - Estar na rede interna não concede mais nada
- **Cada requisição prova identidade e autorização**
 - O perímetro deixa de ser a fronteira de segurança
- **Identidade vira o novo perímetro do sistema**
 - Acesso é avaliado por contexto, não por localização
- **Movimentação lateral do atacante fica muito mais difícil**

O perímetro morreu, a identidade ficou

- Firewall de borda não protege contra ameaça interna
- Um serviço comprometido tem acesso livre na VPC

Zero trust troca a pergunta. Não é de onde vem a requisição, e sim quem é e o que pode. A localização de rede deixa de ser argumento de segurança.

- Service mesh implementa essa troca de forma transparente

Service mesh viabiliza zero trust sem código

- **Sidecar intercepta todo o tráfego do serviço**
 - A aplicação não precisa saber que ele existe
- **mTLS automático entre todos os pods do mesh**
 - Identidade de workload emitida e rotacionada sozinha
- **Políticas de tráfego aplicadas fora da aplicação**
 - Quem fala com quem vira configuração declarativa
- **Observabilidade de segurança vem de brinde no sidecar**

Istio e Vault encerram os incidentes

- **Após os incidentes da seção, zero trust é adotado**
 - Cada serviço prova identidade para falar com outro

LUNALOG

Depois dos incidentes de segurança que marcaram toda esta seção, a LunaLog finalmente implementou zero trust na comunicação interna: cada serviço passou a provar a própria identidade para se comunicar com qualquer outro, independentemente de estar dentro da VPC. O Istio como service mesh habilitou mTLS automático entre todos os pods sem que a equipe tocasse numa linha de código da aplicação. O Vault eliminou de uma vez as variáveis de ambiente com credenciais hardcoded de todos os deploys. Roberto, que um ano antes guardava chave em texto simples, agora liderava uma arquitetura onde nenhum segredo vivia fora do cofre. A LunaLog tinha amadurecido.

Secrets management em ambientes containerizados

- **Segredo nunca vive em imagem, código ou env estático**
 - Um cofre central guarda, injeta e rotaciona segredos
- **Injeção em runtime evita segredo no histórico do git**
 - A aplicação busca o segredo, não o carrega embutido
- **Rotação automática limita a janela de exposição**
 - Se vazar, o segredo expira sozinho em pouco tempo
- **Acesso ao cofre é auditado por identidade da carga**

Arquitetura de segurança da seção 4

- ✓ Criptografia em trânsito, repouso e por envelope, chave no KMS
- ✓ Identidade clara: autenticação, autorização, OAuth, OIDC, JWT, MFA
- ✓ Controle de acesso com RBAC, ABAC, ReBAC e policy-as-code
- ▶ Threat modeling e secure by design acham a falha antes do código
- ✓ Zero trust e service mesh removem a confiança implícita na rede
- ! Segurança é camada sobre camada, nenhuma protege sozinha



Zero trust, service mesh, secrets management, a arquitetura de segurança da LunaLog finalmente está completa. Mas de nada adianta um sistema seguro e escalável se você não consegue publicar mudanças nele com segurança e velocidade. Quantas vezes por dia você consegue subir uma mudança? E quando um deploy dá errado, em quanto tempo você reverte? A próxima seção mostra que CI/CD não é sobre ferramentas, é sobre a capacidade do sistema de evoluir sem se destruir.

Seção 5 - Boas práticas, CI/CD e estratégias de deploy